

Kubernetes Administration Guide

Complete CKA Practice Tasks & Solutions

Table of Contents

1. [Container Runtime Setup](#)
 2. [Application Troubleshooting](#)
 3. [Vertical Pod Autoscaler](#)
 4. [Gateway Resources](#)
 5. [Helm Operations](#)
 6. [Network Configuration](#)
 7. [RBAC & Service Accounts](#)
 8. [Storage Classes](#)
 9. [ConfigMaps](#)
 10. [Priority Classes](#)
 11. [Troubleshooting Control Plane](#)
 12. [Horizontal Pod Autoscaler](#)
 13. [Traffic Splitting](#)
 14. [ETCD Information](#)
 15. [Certificate Signing Requests](#)
-

Container Runtime Setup

Task

Prepare `node01` for a Kubernetes installation by setting up a container runtime.

Node Credentials:

- Username: `bob`
- Password: `caleston123`

Objectives:

- Install the `cri-docker_0.3.16.3-0.debian.deb` package located in `/root`
- Ensure the `cri-docker` service is running and enabled to start on boot

Solution

1. Access the Node

SSH into `node01` using the provided credentials:

```
ssh bob@node01
```

2. Elevate Privileges

Switch to the root user to execute administrative commands:

```
sudo -i
```

3. Install the Package

Use `dpkg` to install the `.deb` package:

```
dpkg -i /root/cri-docker_0.3.16.3-0.debian.deb
```

4. Manage the Service

Start the service and enable it to launch automatically at boot:

```
systemctl start cri-docker  
systemctl enable cri-docker
```

5. Verification

Confirm the service is active and properly enabled:

Command	Expected Output
<code>systemctl is-active cri-docker</code>	active
<code>systemctl is-enabled cri-docker</code>	enabled

Application Troubleshooting

Task

A new application, `orange`, has been deployed but is failing to start correctly. Identify the root cause and resolve the issue.

Solution

1. Identify the Issue

Inspect the pod details to find the error:

```
kubectl describe po orange
```

Check the `initContainers` section. You will notice a syntax error:

```
Command: sh -c sleeep 2;
```

The command `sleeep` is misspelled, causing the container to fail during initialization.

2. Update the Pod

Edit the pod configuration:

```
kubectl edit po orange
```

- Locate the `initContainers` section
- Correct `sleeep 2;` to `sleep 2;`

- Save and exit the editor

3. Apply the Changes

Since Kubernetes doesn't allow live updates to this field, use the generated temporary manifest:

```
kubectl replace -f /tmp/kubectl-edit-xxxx.yaml --force
```

Note: Replace `xxxx` with the actual random string generated by the edit command.

The `--force` flag ensures the old pod is deleted and recreated immediately.

4. Verification

Check the pod status:

```
kubectl get po orange
```

Vertical Pod Autoscaler

Task

Deploy a VPA named `analytics-vpa` for the `analytics-deployment` in the `default` namespace. The VPA must optimize CPU and memory requests using the `Recreate` update mode.

Solution

Execute the following command:

```
kubectl create -n default -f - <<EOF
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: analytics-vpa
  namespace: default
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: analytics-deployment
  updatePolicy:
    updateMode: "Recreate"
EOF
```

Gateway Resources

Task

Create a Gateway resource with the following specifications:

- Name: `web-gateway`

- Namespace: `nginx-gateway`
- Gateway Class: `nginx`
- Listener: Port 80, Protocol HTTP, Name `http`

Solution

```
kubectl create -n nginx-gateway -f - <<EOF
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: web-gateway
  namespace: nginx-gateway
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      protocol: HTTP
      port: 80
EOF
```

Helm Operations

Task

Update the Helm repository and upgrade the existing `kk-mock1` release in the `kk-ns` namespace to version `18.1.15` .

Solution

1. Locate the Release

List all Helm releases:

```
helm ls -A
```

2. Update the Repository

Update your local Helm chart cache:

```
helm repo update
```

3. Verify Available Versions

Search for the specific chart version:

```
helm search repo kk-mock1/nginx -n kk-ns -l | head -n 30
```

4. Upgrade the Release

Perform the upgrade:

```
helm upgrade kk-mock1 kk-mock1/nginx -n kk-ns --version 18.1.15
```

5. Verify the Upgrade

Check the release status:

```
helm ls -n kk-ns
```

Note: Verify the version number under the `CHART` column.

Network Configuration

Task

Adjust the following network parameters on the system and ensure they persist across reboots:

- `net.ipv4.ip_forward = 1`
- `net.bridge.bridge-nf-call-iptables = 1`

Solution

Set the required sysctl parameters:

```
echo 'net.ipv4.ip_forward = 1' >> /etc/sysctl.conf
echo 'net.bridge.bridge-nf-call-iptables = 1' >> /etc/sysctl.conf
sysctl -p
```

Verify the configuration:

```
sysctl net.ipv4.ip_forward
sysctl net.bridge.bridge-nf-call-iptables
```

RBAC & Service Accounts

Task

Create a new service account named `pvviewer` . Grant this service account access to list all PersistentVolumes in the cluster by creating:

- Cluster role: `pvviewer-role`
- ClusterRoleBinding: `pvviewer-role-binding`
- Pod: `pvviewer` with image `redis` using the service account

Solution

Create Service Account

```
kubectl create serviceaccount pvviewer
```

Create Cluster Role

```
kubectl create clusterrole pvviewer-role --resource=persistentvolumes --verb=list
```

Create ClusterRoleBinding

```
kubectl create clusterrolebinding pvviewer-role-binding \  
  --clusterrole=pvviewer-role \  
  --serviceaccount=default:pvviewer
```

Create Pod

```
apiVersion: v1  
kind: Pod  
metadata:  
  labels:  
    run: pvviewer  
  name: pvviewer  
spec:  
  containers:  
  - image: redis  
    name: pvviewer  
  serviceAccountName: pvviewer
```

Apply the manifest:

```
kubectl apply -f pvviewer-pod.yaml
```

Storage Classes

Task

Create a StorageClass named `rancher-sc` with the following specifications:

- Provisioner: `rancher.io/local-path`
- Volume binding mode: `WaitForFirstConsumer`
- Volume expansion: enabled

Solution

Create manifest file `rancher-sc.yaml` :

```
apiVersion: storage.k8s.io/v1  
kind: StorageClass  
metadata:  
  name: rancher-sc  
provisioner: rancher.io/local-path
```

```
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

Apply the configuration:

```
kubectl apply -f rancher-sc.yaml
```

Verify:

```
kubectl get storageclass rancher-sc -o yaml
```

ConfigMaps

Task

Create a ConfigMap named `app-config` in namespace `cm-namespace` with key-value pairs:

- `ENV=production`
- `LOG_LEVEL=info`

Then modify the existing Deployment `cm-webapp` to use these environment variables.

Solution

Create ConfigMap

```
kubectl create configmap app-config -n cm-namespace \
  --from-literal=ENV=production \
  --from-literal=LOG_LEVEL=info
```

Update Deployment

```
kubectl set env deployment/cm-webapp -n cm-namespace \
  --from=configmap/app-config
```

Priority Classes

Task

Create a PriorityClass named `low-priority` with value `50000`. Modify the pod `lp-pod` in namespace `low-priority` to use this priority class.

Solution

Create PriorityClass

Create manifest `pc.yaml` :

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: low-priority
value: 50000
globalDefault: false
description: "Low priority class"
```

Apply:

```
kubectl apply -f pc.yaml
```

Update Pod

Inspect the existing pod:

```
kubectl get pod lp-pod -n low-priority -o yaml
```

Create updated manifest `lp-pod.yaml` :

```
apiVersion: v1
kind: Pod
metadata:
  name: lp-pod
  namespace: low-priority
spec:
  priorityClassName: low-priority
  containers:
  - name: nginx
    image: nginx
```

Recreate the pod:

```
kubectl delete pod lp-pod -n low-priority
kubectl apply -f lp-pod.yaml
```

Troubleshooting Control Plane

Task 1: Fix kubeconfig

A kubeconfig file `super.kubeconfig` has been created under `/root/CKA` . Fix the configuration issue.

Solution

Open the file and change port `9999` to `6443` :

```
vi /root/CKA/super.kubeconfig
```


Verify:

```
kubectl cluster-info --kubeconfig=/root/CKA/super.kubeconfig
```

Task 2: Scale Deployment

Scale the `nginx-deploy` deployment to 3 replicas and troubleshoot if it doesn't work.

Solution

```
kubectl scale deploy nginx-deploy --replicas=3
```

Check control plane components:

```
kubectl get pods -n kube-system
```

If controller-manager is not running, fix the manifest:

```
sed -i 's/kube-controller-manager/kube-controller-manager/g' \
/etc/kubernetes/manifests/kube-controller-manager.yaml
```

Verify:

```
kubectl get deploy
```

Expected output:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx-deploy	3/3	3	3	6m2s

Horizontal Pod Autoscaler

Task

Create an HPA named `api-hpa` for deployment `api-deployment` in namespace `api`. Scale based on custom metric `requests_per_second` with target average value of 1000. Set min replicas to 1 and max to 20.

Solution

Create `api-hpa.yaml` :

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
  namespace: api
spec:
```

```
scaleTargetRef:
  apiVersion: apps/v1
  kind: Deployment
  name: api-deployment
minReplicas: 1
maxReplicas: 20
metrics:
- type: Pods
  pods:
    metric:
      name: requests_per_second
    target:
      type: AverageValue
      averageValue: "1000"
```

Apply:

```
kubectl create -f api-hpa.yaml
```

Traffic Splitting

Task

Configure `web-route` to split traffic: 80% to `web-service` and 20% to `web-service-v2` .

Solution

```
kubectl create -n default -f - <<EOF
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: web-route
  namespace: default
spec:
  parentRefs:
  - name: web-gateway
    namespace: default
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /
    backendRefs:
    - name: web-service
      port: 80
      weight: 80
    - name: web-service-v2
      port: 80
```

weight: 20

EOF

ETCD Information

Task

Find ETCD information on `cka9412` and write to `/opt/course/p1/etcd-info.txt` :

- Server private key location
- Server certificate expiration date
- Is client certificate authentication enabled

Solution

Access Node and Locate Config

```
ssh cka9412
sudo -i
vim /etc/kubernetes/manifests/etcd.yaml
```

Extract Information

From the manifest:

- Private Key: `--key-file=/etc/kubernetes/pki/etcd/server.key`
- Client Cert Auth: `--client-cert-auth=true`
- Certificate: `--cert-file=/etc/kubernetes/pki/etcd/server.crt`

Check Certificate Expiration

```
openssl x509 -noout -text -in /etc/kubernetes/pki/etcd/server.crt | grep Validity -
A2
```

Create Output File

```
# /opt/course/p1/etcd-info.txt
Server private key location: /etc/kubernetes/pki/etcd/server.key
Server certificate expiration date: Oct 29 14:19:27 2025 GMT
Is client certificate authentication enabled: yes
```

Certificate Signing Requests

Task

Create a new user `john` and grant access to the cluster using a `CertificateSigningRequest`.

Files provided:

- Private key: `/root/CKA/john.key`
- CSR file: `/root/CKA/john.csr`

Requirements:

- Create CSR object: `john-developer`
- Approve the CSR
- Create Role: `developer` in `development` namespace (permissions: create, list, get, update, delete pods)
- Create RoleBinding: `developer-role-binding`
- Verify access

Solution

Step 1: Create CSR Object

Generate base64 value:

```
base64 < /root/CKA/john.csr | tr -d '\n'
```

Create `john-csr.yaml` :

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: john-developer
spec:
  request: <PASTE_BASE64_OUTPUT_HERE>
  signerName: kubernetes.io/kube-apiserver-client
  usages:
    - digital signature
    - key encipherment
    - client auth
```

Apply:

```
kubectl apply -f john-csr.yaml
```

Step 2: Approve CSR

```
kubectl certificate approve john-developer
```

Verify:

```
kubectl get csr john-developer
```

Expected: `Approved, Issued`

Step 3: Extract Certificate

```
kubectl get csr john-developer -o jsonpath='{.status.certificate}' \
| base64 -d > /root/CKA/john.crt
```

Step 4: Create Role

```
kubectl create role developer \
  --resource=pods \
  --verb=create,list,get,update,delete \
  --namespace=development
```

Step 5: Create RoleBinding

```
kubectl create rolebinding developer-role-binding \
  --role=developer \
  --user=john \
  --namespace=development
```

Step 6: Verify Permissions

Test in development namespace:

```
kubectl auth can-i create pods --as=john -n development
```

Expected: `yes`

Test outside namespace:

```
kubectl auth can-i create pods --as=john -n default
```

Expected: `no`

Additional Tasks

Helm Chart Validation and Upgrade

Task

Validate and install a new Helm chart as `webpage-server-02` , then uninstall the old release `webpage-server-01` .

Solution

List current releases:

```
helm ls -n default
```

Validate the new chart:

```
cd /root/
helm lint ./new-version
```

Install new release:

```
helm install webpage-server-02 ./new-version
```

Uninstall old release:

```
helm uninstall webpage-server-01 -n default
```

Pod Network CIDR

Task

Identify the Pod subnet configured for the cluster and output to `/root/pod-cidr.txt`.

Solution

Inspect kubeadm configuration:

```
kubectl -n kube-system get configmap kubeadm-config -o yaml | grep podSubnet
```

Save to file:

```
kubectl -n kube-system get configmap kubeadm-config -o yaml \
  | awk '/podSubnet:/{print $2}' > /root/pod-cidr.txt
```

Verify:

```
cat /root/pod-cidr.txt
# Expected: 172.17.0.0/16
```

Important Note:

- Cluster PodCIDR (entire pool): `172.17.0.0/16`
 - Find with: `kubectl get cm kubeadm-config -n kube-system -o yaml`
- Node PodCIDR (single slice): `172.17.0.0/24`
 - Find with: `kubectl get node <name> -o jsonpath='{.spec.podCIDR}'`

Service CIDR Change

Task

Change the Cluster Service CIDR to `11.96.0.0/12` and verify new services receive IPs from the new range.

Solution

Step 1: Initial Setup

```
kubectl run check-ip --image=httpd:2-alpine
kubectl expose pod check-ip --name check-ip-service --port 80
# Note the old IP (e.g., 10.97.6.41)
```

Step 2: Update Control Plane Manifests

Update `--service-cluster-ip-range=11.96.0.0/12` in:

- `/etc/kubernetes/manifests/kube-apiserver.yaml`
- `/etc/kubernetes/manifests/kube-controller-manager.yaml`

Wait for components to restart:

```
watch kubectl -n kube-system get pods
```

Step 3: Update ServiceCIDR Resource

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: ServiceCIDR
metadata:
  name: svc-cidr-new
spec:
  cidrs:
  - 11.96.0.0/12
EOF
```

Delete old ServiceCIDR:

```
kubectl delete servicecidr kubernetescidr
```

Step 4: Verification

```
kubectl expose pod check-ip --name check-ip-service2 --port 80
kubectl get svc
```

Expected: `check-ip-service2` should have an IP starting with `11.x.x.x`

Kube-Proxy iptables

Task

Verify kube-proxy functionality in namespace `project-hamster` by creating a Pod and Service, then document the iptables rules on node `cka3962`.

Solution

Access node and create resources:

```
ssh cka3962

# Create Pod
kubectl -n project-hamster run p2-pod --image=nginx:1-alpine

# Create Service
```

```
kubectl -n project-hamster expose pod p2-pod --name p2-service --port 3000 --target-port 80
```

Identify iptables rules:

```
sudo -i  
iptables-save | grep p2-service
```

Save rules:

```
iptables-save | grep p2-service > /opt/course/p2/iptables.txt
```

Cleanup and verify:

```
kubectl -n project-hamster delete svc p2-service  
iptables-save | grep p2-service # Should return no results
```

Key CKA Points to Remember

Certificate Signing Requests

- `signerName` is mandatory in Kubernetes ≥ 1.19
- CSR must be base64 encoded (single line)
- CSR CN must match user (CN=john)
- Role + RoleBinding must be in the target namespace
- Role (not ClusterRole) limits access to namespace

Workflow for CSR

1. CSR → Approve → Extract Cert
2. Create Role → Create RoleBinding
3. Test with `can-i`

Common Troubleshooting Steps

1. Check pod status: `kubectl get pods`
2. Describe resources: `kubectl describe <resource> <name>`
3. Check logs: `kubectl logs <pod-name>`
4. Verify control plane: `kubectl get pods -n kube-system`
5. Check manifests: `/etc/kubernetes/manifests/`

Important File Locations

- Static pod manifests: `/etc/kubernetes/manifests/`
- PKI certificates: `/etc/kubernetes/pki/`
- Kubeconfig: `~/.kube/config` or `/etc/kubernetes/admin.conf`
- Kubelet config: `/var/lib/kubelet/config.yaml`

Conclusion

This guide covers essential Kubernetes administration tasks commonly found in CKA exams. Practice these scenarios in a lab environment to build muscle memory for the exam.

Tips for Success:

- Always verify your changes
- Use `kubectl` imperative commands when possible for speed
- Know how to troubleshoot control plane components
- Understand RBAC thoroughly
- Practice time management during the exam

Document Version: 1.0

Last Updated: February 2026

Target Audience: CKA Exam Candidates